Vertex AI：超參數調整

來源：https://codelabs.developers.google.com/vertex_hyperparameter_tuning?hl=zh-tw

步驟數：7

目錄

1. 1. 總覽
2. 2. Vertex AI 簡介
3. 3. 設定環境
4. 4. 將訓練應用程式程式碼容器化
5. 5. 在 Vertex AI 上執行超參數調整工作
6. 6. [選用] 使用 Vertex SDK
7. 7. 清除所用資源

步驟 1 · 約 1 分鐘

1. 總覽

在本實驗室中，您將使用 Vertex AI 執行 TensorFlow 模型的超參數調整工作。雖然本實驗室使用 TensorFlow 撰寫模型程式碼，但這些概念也適用於其他機器學習架構。

課程內容

內容如下：

- 修改訓練應用程式程式碼，進行自動超參數調整
- 透過 Vertex AI 使用者介面設定及啟動超參數調整工作
- 使用 Vertex AI Python SDK 設定及啟動超參數調整工作

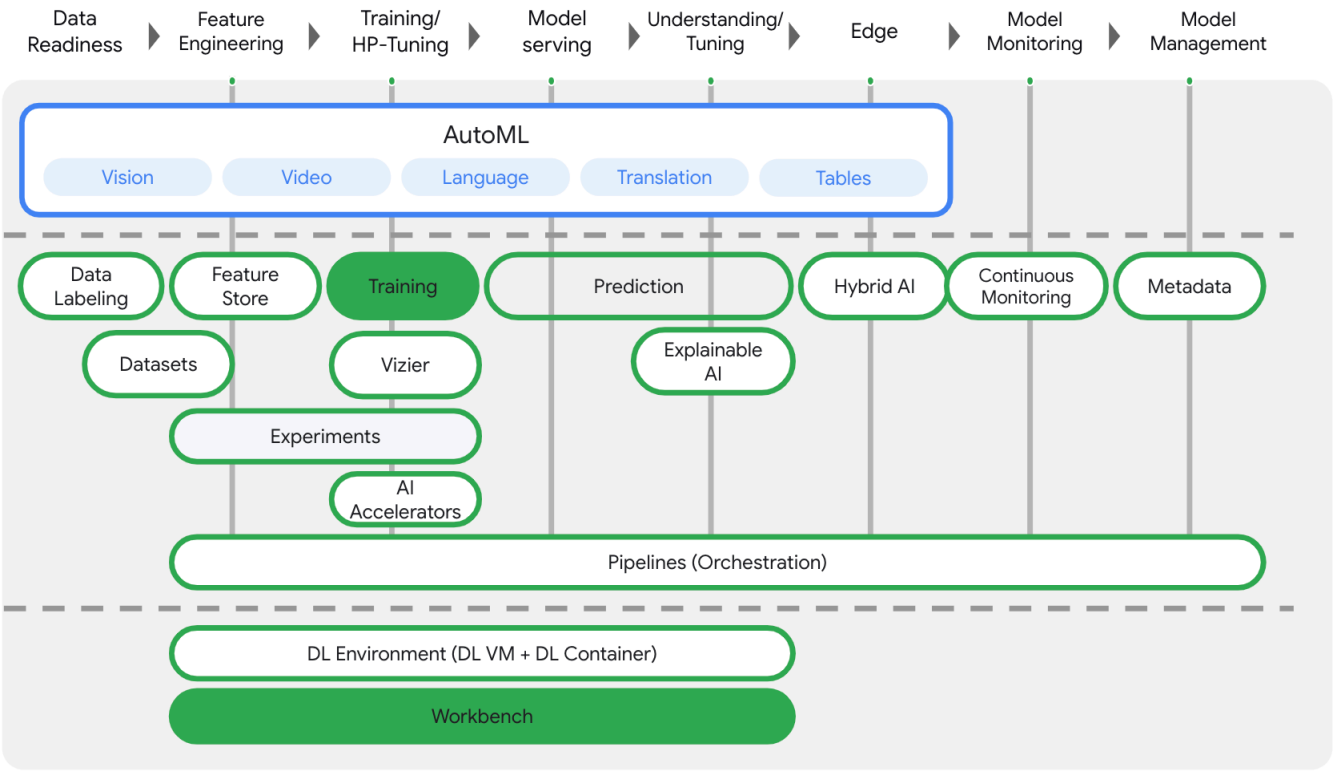
在 Google Cloud 上執行這個實驗室的總費用約為 **$3** 美元。

步驟 2 · 約 2 分鐘

2. Vertex AI 簡介

本實驗室使用 Google Cloud 最新推出的 AI 產品服務。Vertex AI 整合了 Google Cloud 機器學習服務，提供流暢的開發體驗。以 AutoML 訓練的模型和自訂模型，先前需透過不同的服務存取。這項新服務將兩者併至單一 API，並加入其他新產品。您也可以將現有專案遷移至 Vertex AI。如有任何意見，請參閱支援頁面。

Vertex AI 包含許多不同的產品，可支援端對端機器學習工作流程。本實驗室將著重於下列產品：**訓練**和 **Workbench**。



步驟 3 · 約 10 分鐘

3. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照這裡的說明操作。

步驟 1：啟用 Compute Engine API

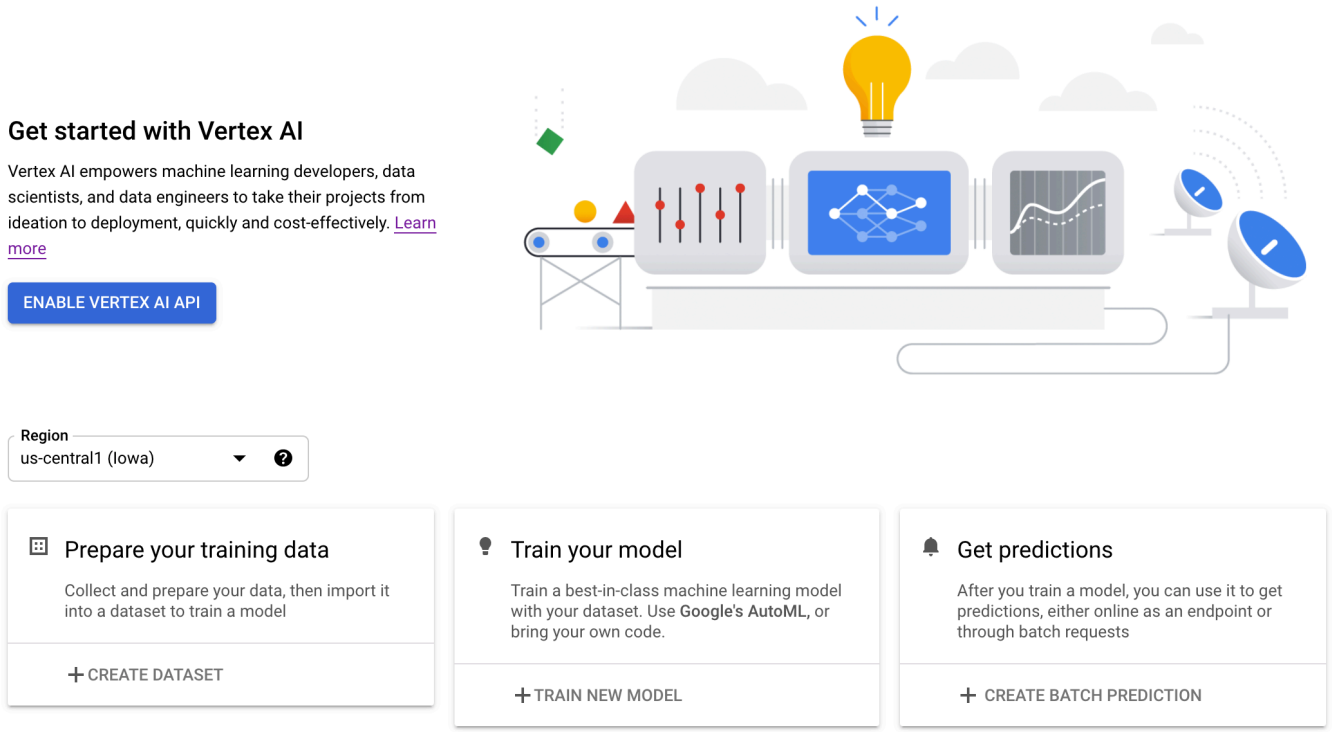
前往「Compute Engine」，然後選取「啟用」 (如果尚未啟用)。您需要這項資訊才能建立筆記本執行個體。

步驟 2：啟用 Container Registry API

前往「Container Registry」，然後選取「啟用」(如果尚未啟用)。您將使用這個工具為自訂訓練工作建立容器。

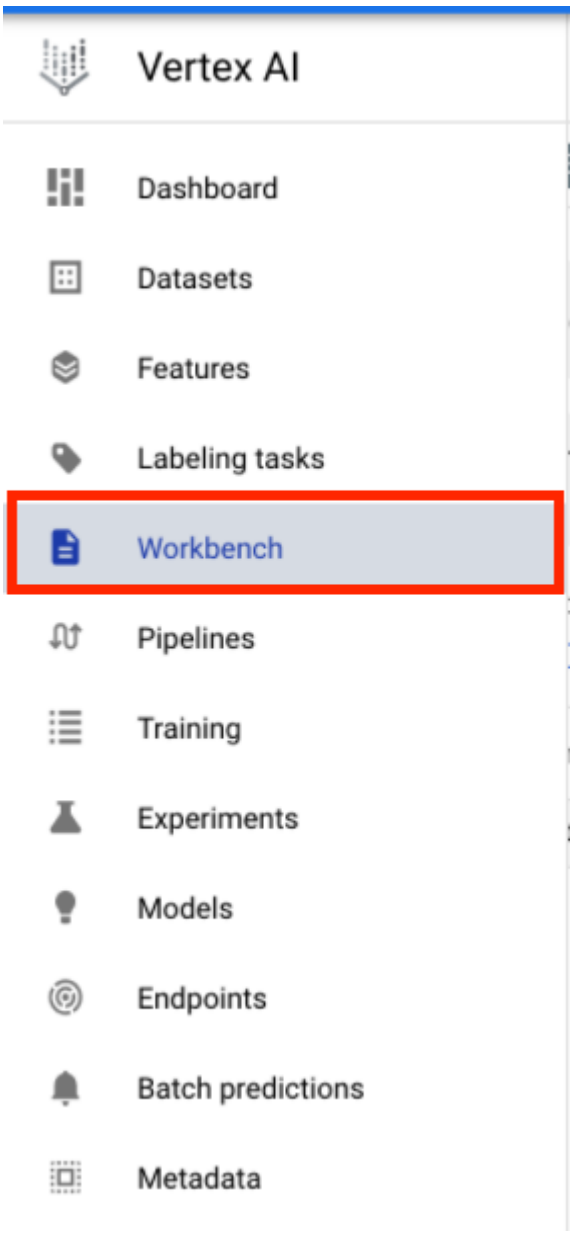
步驟 3：啟用 Vertex AI API

前往 Cloud 控制台的 Vertex AI 專區，然後點選「啟用 Vertex AI API」。

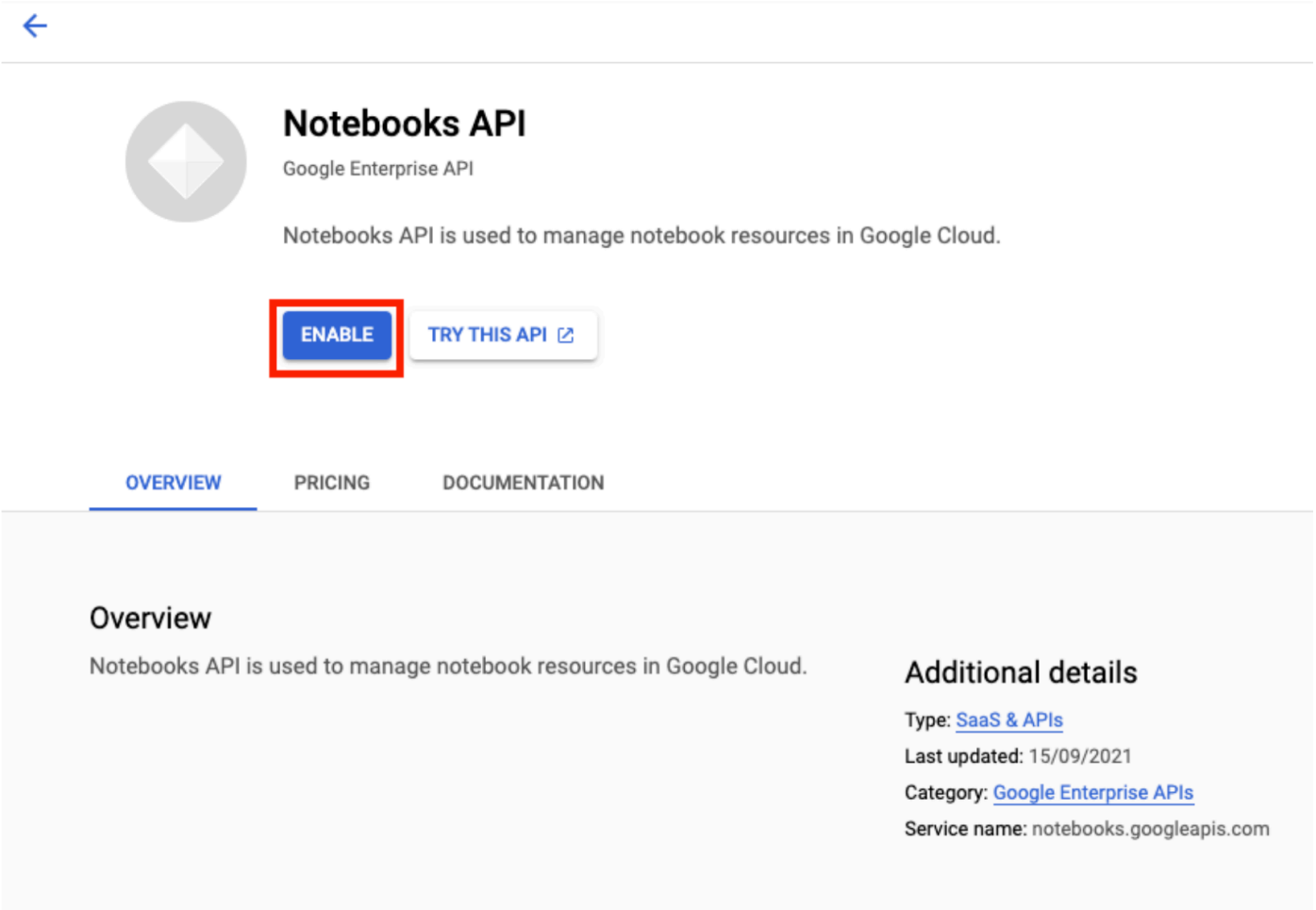


步驟 4：建立 Vertex AI Workbench 執行個體

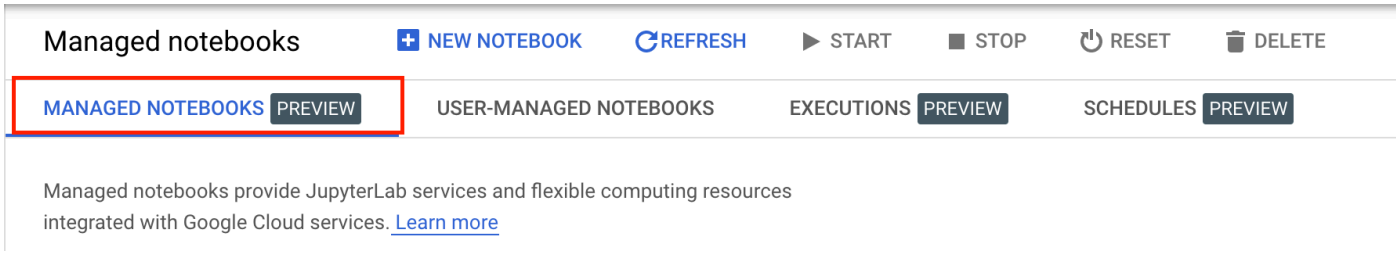
在 Cloud 控制台的「Vertex AI」部分中，按一下「Workbench」：



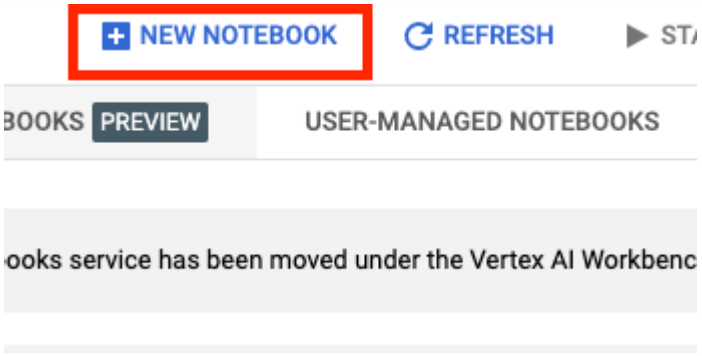
如果尚未啟用 Notebooks API，請先啟用。



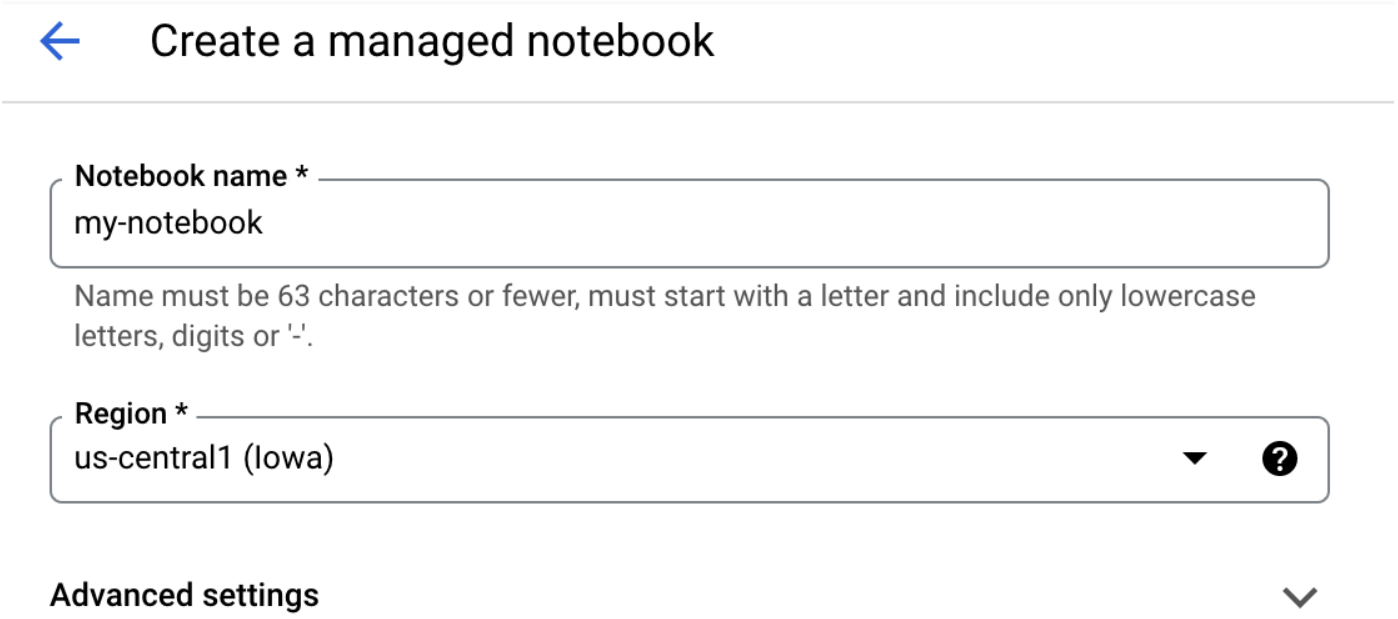
啟用後，按一下「MANAGED NOTEBOOKS」(代管型筆記本)：



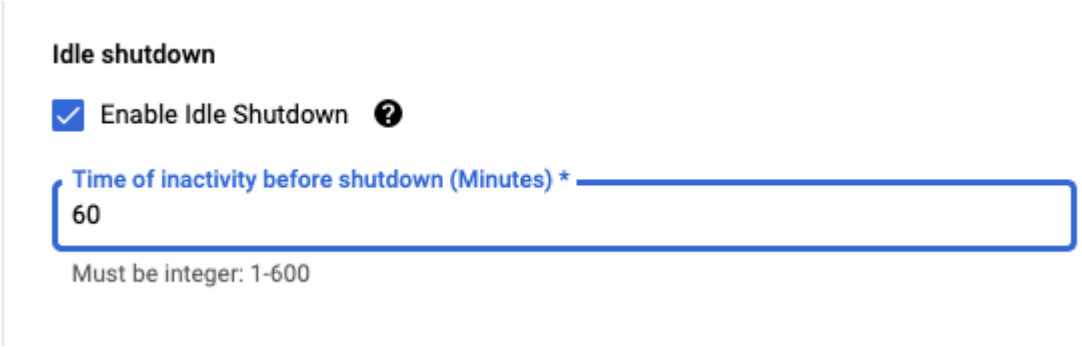
然後選取「新增記事本」。



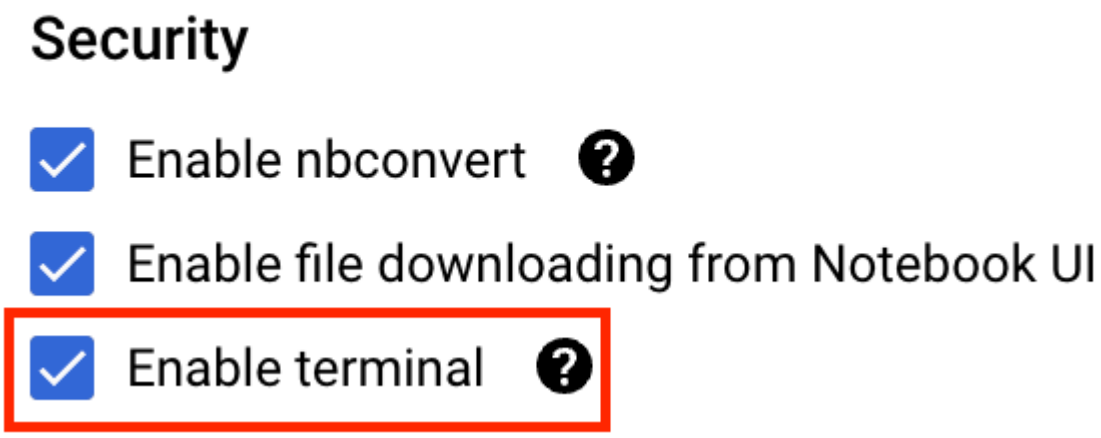
為筆記本命名，然後按一下「進階設定」。



在「進階設定」下方啟用閒置關機功能，並將分鐘數設為 60。也就是說，筆記本會在閒置時自動關機，避免產生不必要的費用。



在「安全性」下方，選取「啟用終端機」(如果尚未啟用)。

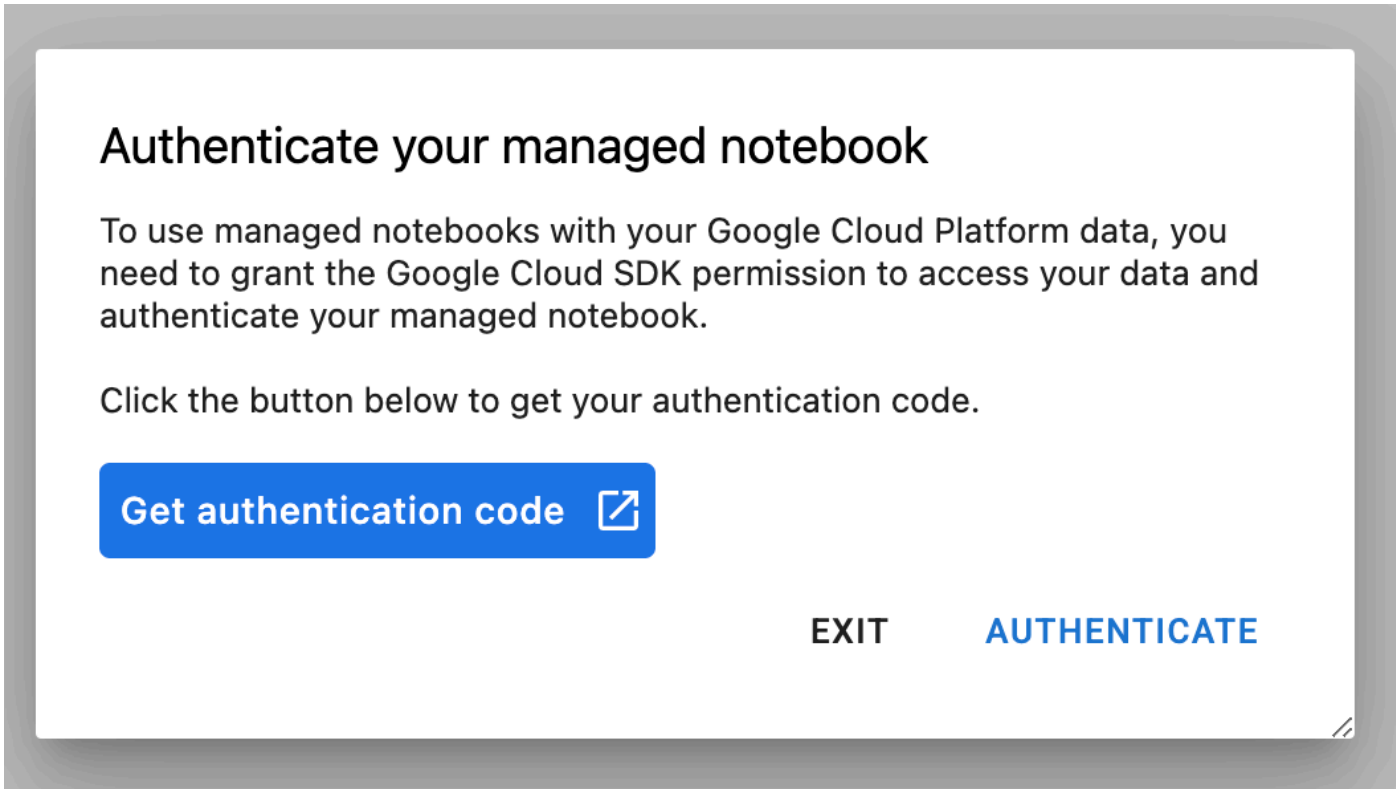


其他進階設定可以保留原樣。

接著點選「建立」。執行個體會在幾分鐘內佈建完畢。

建立執行個體後，請選取「Open JupyterLab」。

首次使用新執行個體時，系統會要求您驗證身分。按照 UI 中的步驟操作。



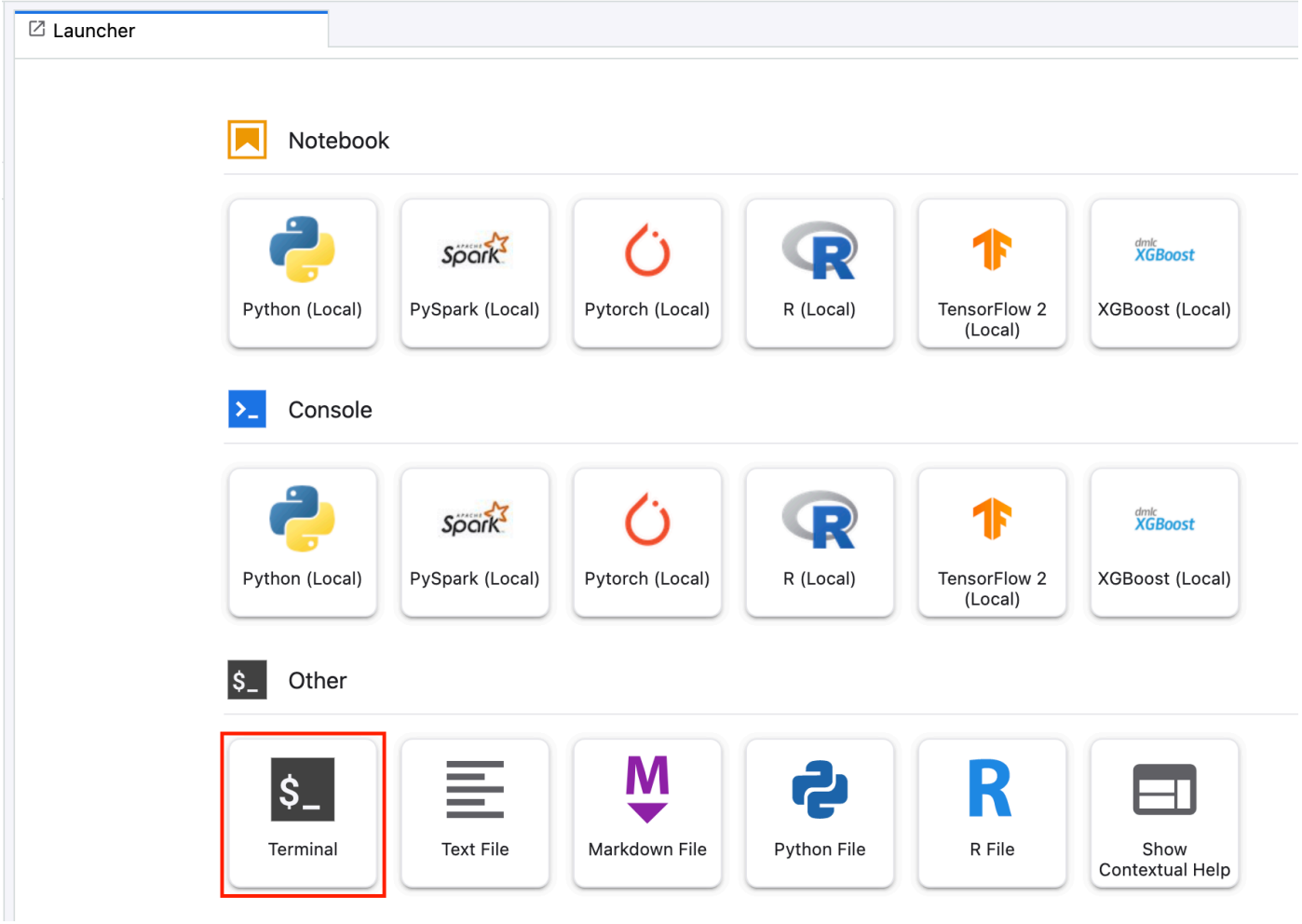
步驟 4 · 約 5 分鐘

4. 將訓練應用程式程式碼容器化

在本實驗室中，您將訓練及調整圖像分類模型，該模型是使用 TensorFlow Datasets 中的馬匹或人類資料集訓練而成。

您要將訓練應用程式程式碼放入 Docker 容器，並將這個容器推送至 Google Container Registry，藉此將超參數調整工作提交至 Vertex AI。使用這種方法，您可以調整以任何架構建構的模型超參數。

首先，請從 Launcher 選單開啟筆記本執行個體中的終端機視窗：



建立名為 `horses_or_humans` 的新目錄，然後切換至該目錄：

```
mkdir horses_or_humans
cd horses_or_humans
```

步驟 1：建立 Dockerfile

將程式碼容器化的第一步是建立 Dockerfile。Dockerfile 包含執行映像檔所需的所有指令。這會安裝所有必要的程式庫 (包括 CloudML Hypertune 程式庫)，並設定訓練程式碼的進入點。

在終端機中建立空白的 Dockerfile：

```
touch Dockerfile
```

開啟 Dockerfile，然後將下列內容複製到其中：

```
FROM gcr.io/deeplearning-platform-release/tf2-gpu.2-7

WORKDIR /

# Installs hypertune library
RUN pip install cloudml-hypertune

# Copies the trainer code to the docker image.
COPY trainer /trainer

# Sets up the entry point to invoke the trainer.
ENTRYPOINT ["python", "-m", "trainer.task"]
```

這個 Dockerfile 使用 深度學習容器 TensorFlow 企業版 2.7 GPU Docker 映像檔。Google Cloud 上的深度學習容器預先安裝了許多常見的機器學習和資料科學框架。下載該映像檔後，這個 Dockerfile 會設定訓練程式碼的進入點。您尚未建立這些檔案，在下一個步驟中，您將新增用於訓練及調整模型的程式碼。

步驟 2：新增模型訓練程式碼

在終端機中執行下列指令，為訓練程式碼建立目錄，以及新增程式碼的 Python 檔案：

```
mkdir trainer
touch trainer/task.py
```

`horses_or_humans/` 目錄現在應該包含下列項目：

```
+ Dockerfile
+ trainer/
    + task.py
```

接著，開啟剛才建立的 `task.py` 檔案，然後複製下列程式碼。

```
import tensorflow as tf
import tensorflow_datasets as tfds
import argparse
import hypertune

NUM_EPOCHS = 10


def get_args():
  '''Parses args. Must include all hyperparameters you want to tune.'''

  parser = argparse.ArgumentParser()
  parser.add_argument(
      '--learning_rate',
      required=True,
      type=float,
      help='learning rate')
  parser.add_argument(
      '--momentum',
      required=True,
      type=float,
      help='SGD momentum value')
  parser.add_argument(
      '--num_units',
      required=True,
      type=int,
      help='number of units in last hidden layer')
  args = parser.parse_args()
  return args


def preprocess_data(image, label):
  '''Resizes and scales images.'''

  image = tf.image.resize(image, (150,150))
  return tf.cast(image, tf.float32) / 255., label


def create_dataset():
  '''Loads Horses Or Humans dataset and preprocesses data.'''

  data, info = tfds.load(name='horses_or_humans', as_supervised=True, with_info=True)

  # Create train dataset
  train_data = data['train'].map(preprocess_data)
  train_data  = train_data.shuffle(1000)
  train_data  = train_data.batch(64)

  # Create validation dataset
  validation_data = data['test'].map(preprocess_data)
  validation_data  = validation_data.batch(64)
```

```
  return train_data, validation_data


def create_model(num_units, learning_rate, momentum):
  '''Defines and compiles model.'''

  inputs = tf.keras.Input(shape=(150, 150, 3))
  x = tf.keras.layers.Conv2D(16, (3, 3), activation='relu')(inputs)
  x = tf.keras.layers.MaxPooling2D((2, 2))(x)
  x = tf.keras.layers.Conv2D(32, (3, 3), activation='relu')(x)
  x = tf.keras.layers.MaxPooling2D((2, 2))(x)
  x = tf.keras.layers.Conv2D(64, (3, 3), activation='relu')(x)
  x = tf.keras.layers.MaxPooling2D((2, 2))(x)
  x = tf.keras.layers.Flatten()(x)
  x = tf.keras.layers.Dense(num_units, activation='relu')(x)
  outputs = tf.keras.layers.Dense(1, activation='sigmoid')(x)
  model = tf.keras.Model(inputs, outputs)
  model.compile(
      loss='binary_crossentropy',
      optimizer=tf.keras.optimizers.SGD(learning_rate=learning_rate, momentum=momentum),
      metrics=['accuracy'])
  return model


def main():
  args = get_args()
  train_data, validation_data = create_dataset()
  model = create_model(args.num_units, args.learning_rate, args.momentum)
  history = model.fit(train_data, epochs=NUM_EPOCHS, validation_data=validation_data)

  # DEFINE METRIC
  hp_metric = history.history['val_accuracy'][-1]

  hpt = hypertune.HyperTune()
  hpt.report_hyperparameter_tuning_metric(
      hyperparameter_metric_tag='accuracy',
      metric_value=hp_metric,
      global_step=NUM_EPOCHS)


if __name__ == "__main__":
    main()
```

建構容器前，請先深入瞭解程式碼。使用超參數調整服務時，有幾項專屬元件。

1. 指令碼會匯入 `hypertune` 程式庫。請注意，步驟 1 的 Dockerfile 包含使用 pip 安裝這個程式庫的指令。
2. 函式 `get_args()` 會為您要調整的每個超參數定義指令列引數。在本範例中，要調整的超參數是學習率、最佳化工具中的動量值，以及模型最後一個隱藏層中的單元數量，但您也可以嘗試其他超參數。然後，系統會使用這些引數中傳遞的值，在程式碼中設定對應的超參數。
3. 在 `main()` 函式結尾，系統會使用 `hypertune` 程式庫定義要最佳化的指標。在 TensorFlow 中，keras `model.fit` 方法會傳回 `History` 物件。`History.history` 屬性會記錄連續訓練週期的訓練損失值和指標值。如果將驗證資料

傳遞至 `model.fit`，`History.history` 屬性也會包含驗證損失和指標值。舉例來說，如果您使用驗證資料訓練模型三個訓練週期，並提供 `accuracy` 做為指標，則 `History.history` 屬性會類似於下列字典。

```
{
 "accuracy": [
   0.7795261740684509,
   0.9471358060836792,
   0.9870933294296265
 ],
 "loss": [
   0.6340447664260864,
   0.16712145507335663,
   0.04546636343002319
 ],
 "val_accuracy": [
   0.3795261740684509,
   0.4471358060836792,
   0.4870933294296265
 ],
 "val_loss": [
   2.044623374938965,
   4.100203514099121,
   3.0728273391723633
 ]
```

如要讓超參數調整服務找出可將模型驗證準確率提升至最高的超參數值，請將指標定義為 `val_accuracy` 清單的最後一個項目 (或 `NUM_EPOCS - 1`)。然後將這項指標傳遞至 `HyperTune` 的執行個體。您可以為 `hyperparameter_metric_tag` 選擇任何字串，但稍後啟動超參數調整工作時，需要再次使用該字串。

步驟 3：建構容器

在終端機中執行下列指令，為專案定義環境變數，並將 `your-cloud-project` 替換為專案 ID：

> 您可以在終端機中執行 `gcloud config list --format 'value(core.project)'` 來取得專案 ID

```
PROJECT_ID='your-cloud-project'
```

在 Google Container Registry 中，使用容器映像檔的 URI 定義變數：

```
IMAGE_URI="gcr.io/$PROJECT_ID/horse-human:hypertune"
```

設定 Docker

```
gcloud auth configure-docker
```

接著，從 `horses_or_humans` 目錄的根層級執行下列指令，建構容器：

```
docker build ./ -t $IMAGE_URI
```

最後，將其推送至 Google Container Registry：

```
docker push $IMAGE_URI
```

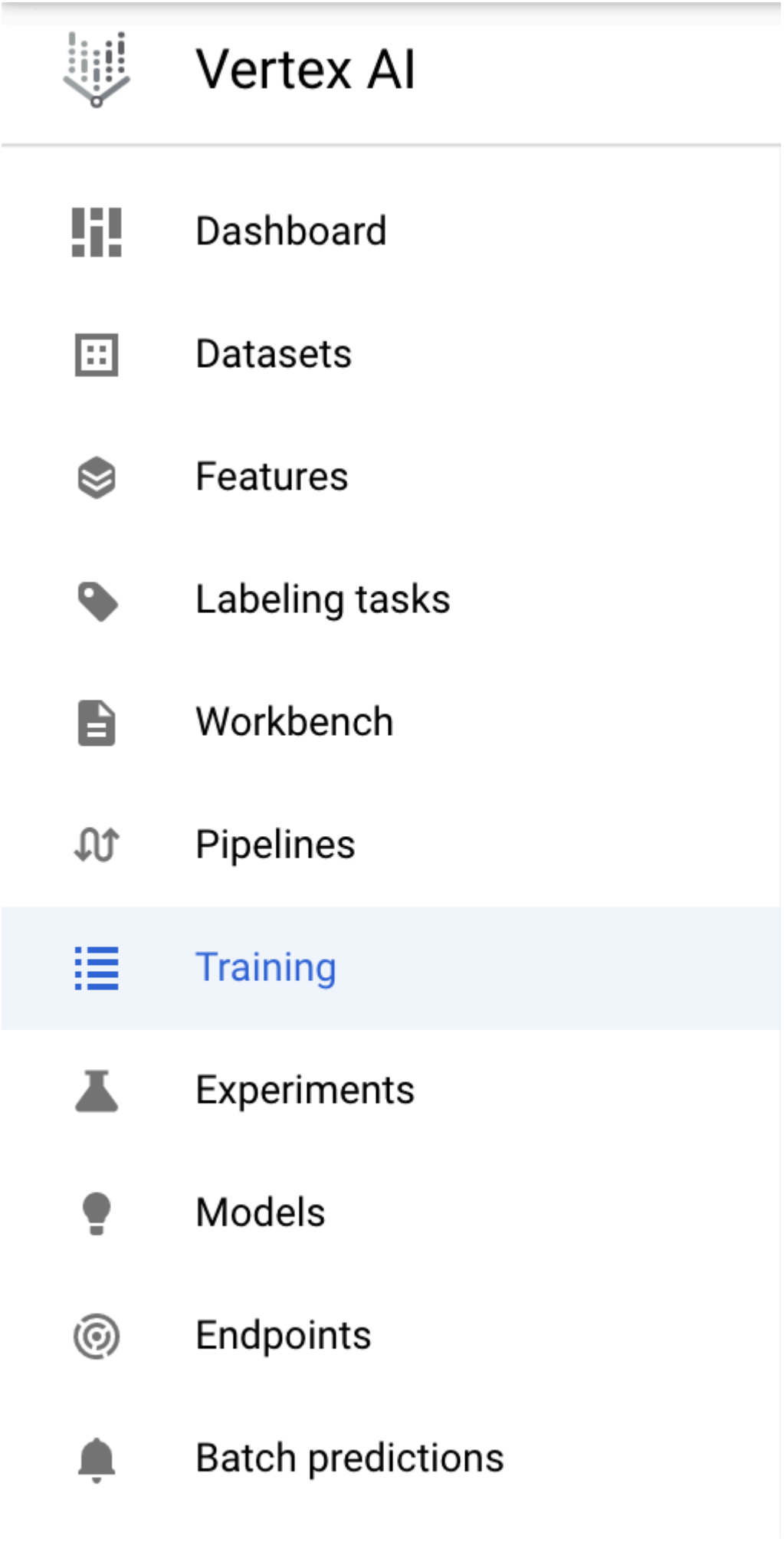
將容器推送至 Container Registry 後，您就可以啟動自訂模型超參數調整工作。

步驟 5 · 約 50 分鐘

5. 在 Vertex AI 上執行超參數調整工作

本實驗室使用 Google Container Registry 上的自訂容器，透過自訂訓練執行作業，但您也可以使用 Vertex AI 預先建構的容器執行超參數調整工作。

首先，請前往 Cloud 控制台的 Vertex 專區，然後點選「訓練」部分：


Vertex AI
Dashboard
Datasets
Features
Labeling tasks
Workbench
Pipelines
Training
Experiments
Models
Endpoints
Batch predictions

Metadata

步驟 1：設定訓練工作

按一下「建立」，輸入超參數調整工作的參數。

- 在「Dataset」(資料集) 下方，選取「No managed dataset」(沒有代管資料集)
- 然後選取「Custom training (advanced)」(自訂訓練 (進階)) 做為訓練方法，並點按「Continue」(繼續)。
- 在「Model name」(模型名稱) 中輸入 `horses-humans-hyptertune` (或您想為模型命名的任何名稱)
- 按一下 [Continue] (繼續)。

在「容器設定」步驟中，選取「自訂容器」：

Select a pre-built container or build a custom container using ML frameworks (as well as non-ML dependencies, libraries and binaries) that are not otherwise supported.
Learn more

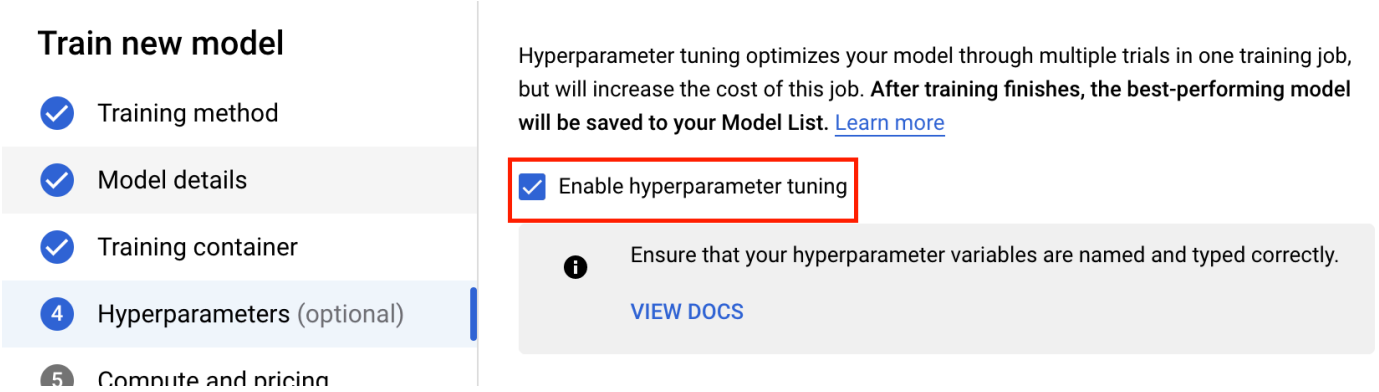
Pre-built container
View the list of supported runtimes including TensorFlow, scikit-learn and PyTorch versions

Custom container
Build a custom Docker container. Must be stored in Container Registry

在第一個方塊 (「容器映像檔」) 中，輸入上一節的 `IMAGE_URI` 變數值。應為： `gcr.io/your-cloud-project/horse-human:hypertune` ，並換成您自己的專案名稱。其餘欄位保留空白，然後點選「繼續」。

步驟 2：設定超參數調整工作

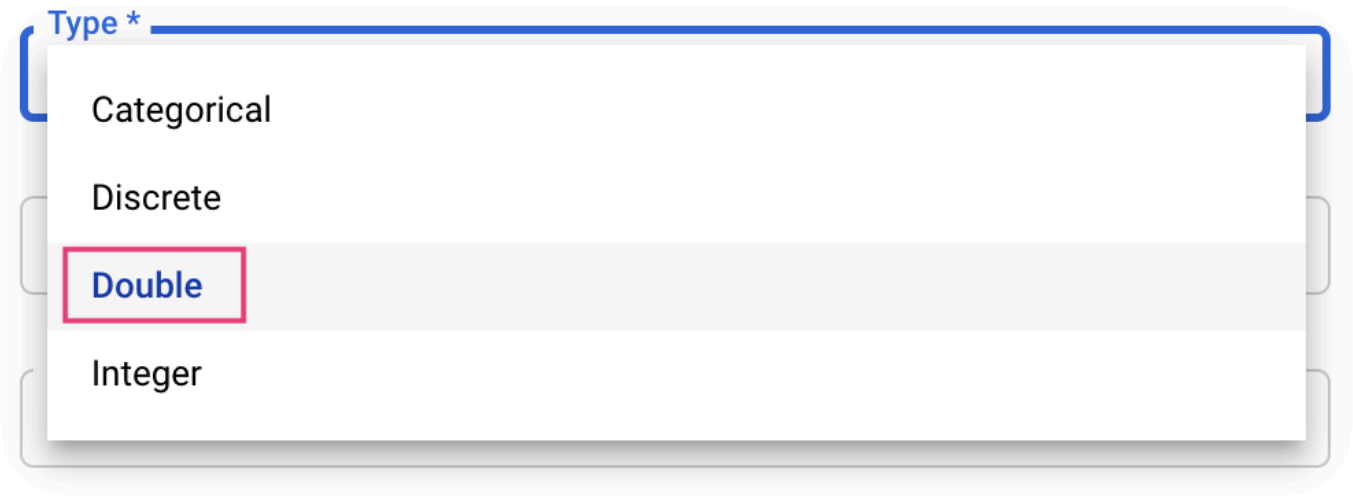
選取「Enable hyperparameter tuning」(啟用超參數微調)。



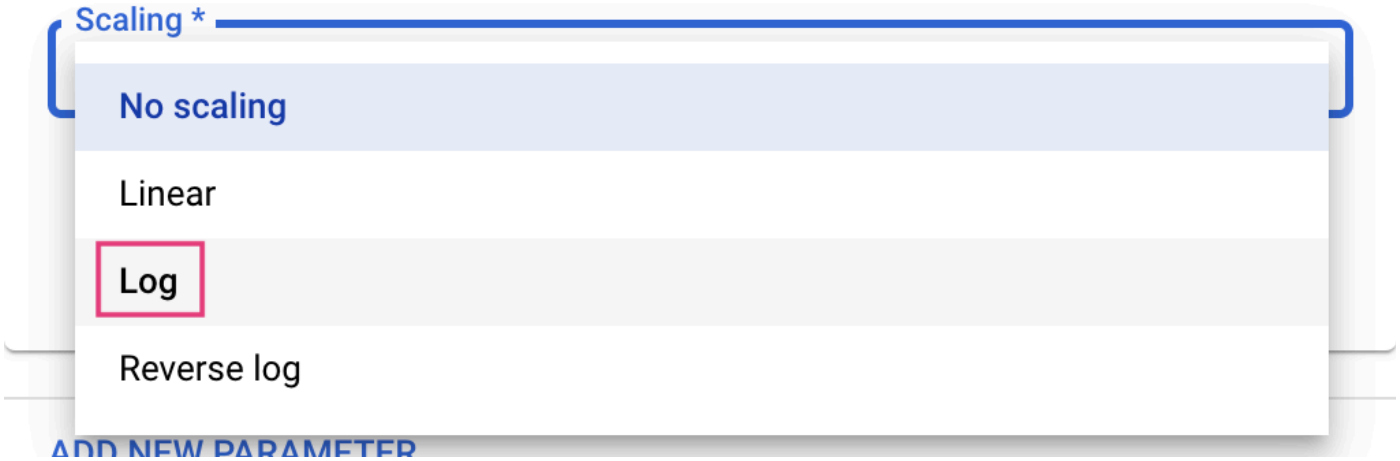
設定超參數

接著，您需要在訓練應用程式程式碼中，加入您設定為指令列引數的超參數。新增超參數時，您必須先提供名稱。這應與您傳遞至 `argparse` 的引數名稱相符。

接著，您將選取「類型」和微調服務嘗試的值範圍。如果選取「Double」或「Integer」類型，則必須提供最小值和最大值。如果選取「類別」或「離散」，則必須提供值。



如果是「Double」和「Integer」類型，您也需要提供「Scaling」值。



新增 `learning_rate` 超參數後，請新增 `momentum` 和 `num_units` 的參數。

Parameter name *
num_units

Type *
Discrete

Values *
64,128,512

Enter all the values that you want to tune, separated with commas.

Scaling *
No scaling

設定指標

加入超參數後，接下來請提供要最佳化的指標和目標。這應該與您在訓練應用程式中設定的 `hyperparameter_metric_tag` 相同。

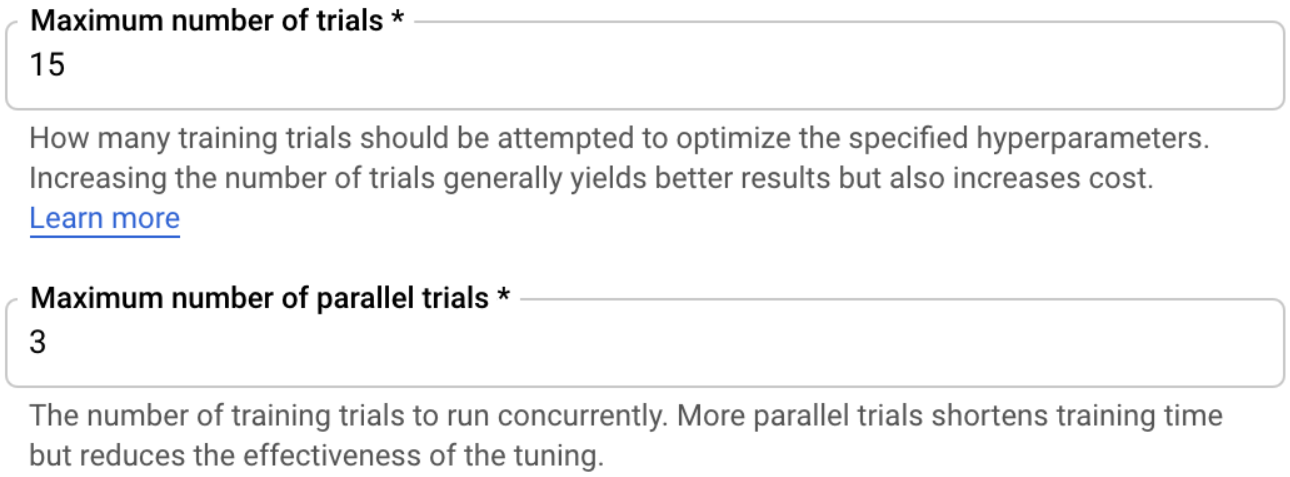
Metric to optimize *
accuracy

Goal *
Maximize

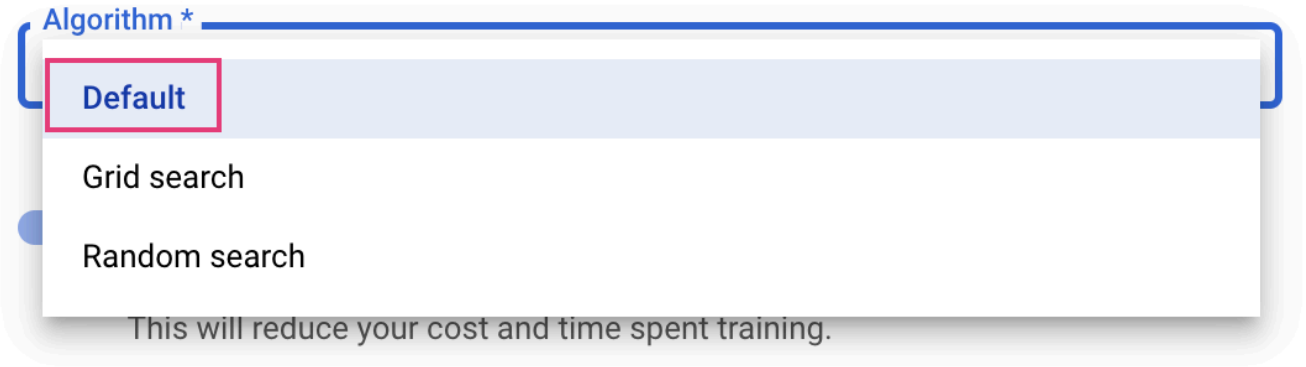
Vertex AI 超參數調整服務會使用您在上一個步驟中設定的值，多次執行訓練應用程式的試驗。您必須為服務執行的測試次數設定上限。一般來說，測試次數越多，結果越好，但當測試次數達到某個程度後，再增加測試次數對您想最佳化的指標就幾乎沒有影響。建議您先進行少量測試，瞭解所選超參數的影響程度，再擴大規模進行大量測試。

您也需要設定並行測試次數上限。增加平行測試次數可縮短超參數微調工作的執行時間，但整體而言，工作成效可能會降低。這是因為預設的微調策略會根據先前的試驗結果，在後續試驗中指派值。如果同時進行太多試驗，有些試驗開始時，可能無法參考仍在進行中的試驗結果。

為方便示範，您可以將試驗數量設為 15，並將平行試驗數量上限設為 3。您可以嘗試不同的數字，但這可能會導致調整時間變長，費用也隨之提高。



最後一個步驟是選取「Default」做為搜尋演算法，這會使用 Google Vizier 執行貝氏最佳化，以調整超參數。如要進一步瞭解這項演算法，請參閱這篇文章。



按一下「繼續」。

步驟 3：設定運算資源

在「Compute and pricing」(運算和價格) 中，保留選取的區域，並依下列方式設定「Worker pool 0」(工作站集區 0)。

Worker pool 0

Machine type *
n1-standard-4, 4 vCPUs, 15 GiB memory

Accelerator type
NVIDIA_TESLA_V100

Accelerators can speed up model training that involves intensive compute tasks. Learn more

Accelerator count
1

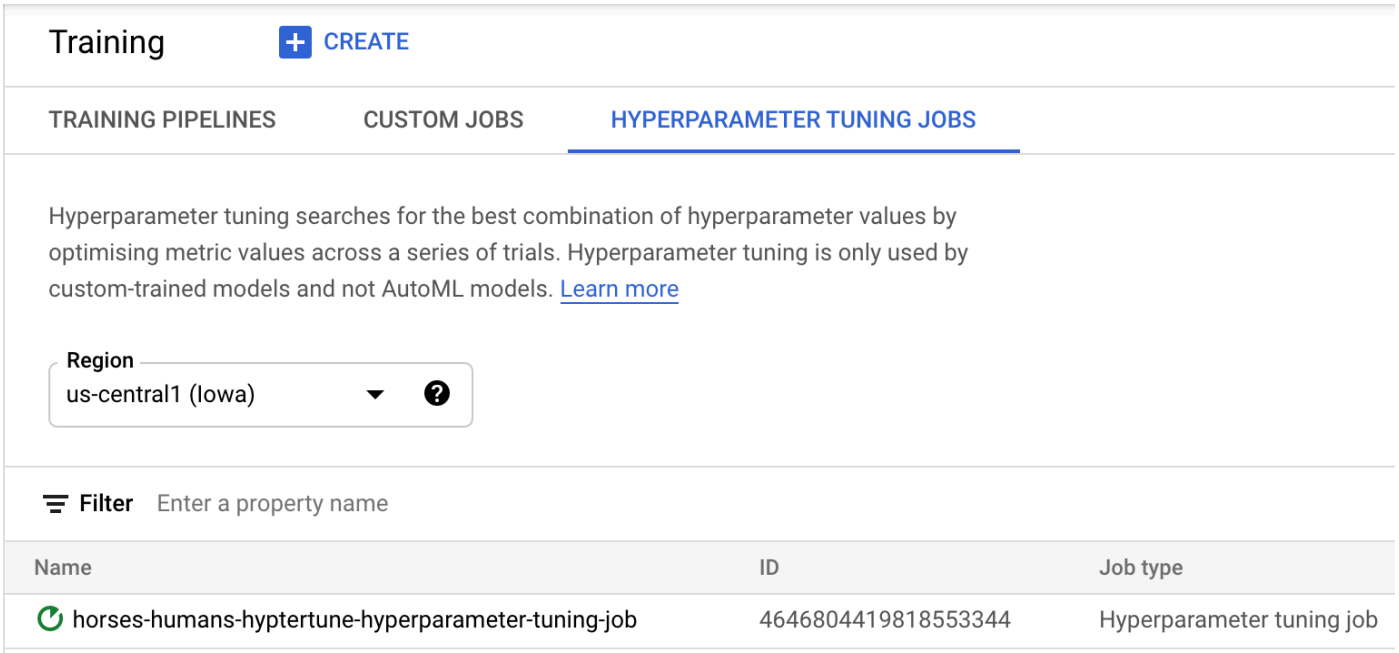
Worker count
1

Disk type
SSD

Disk size (GB)
100

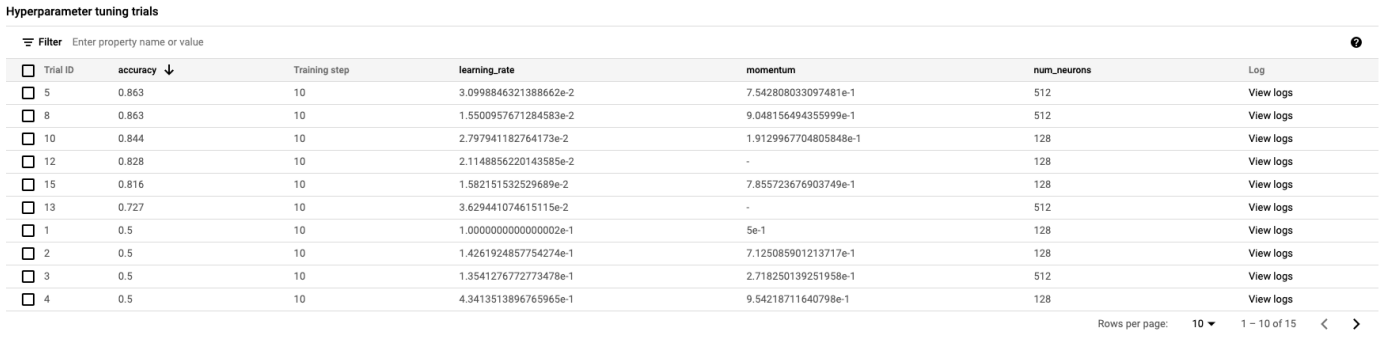
> **注意**：您可以選擇是否要使用 GPU。如果不使用 GPU，超參數調整工作只會稍微延長完成時間。

按一下「開始訓練」，啟動超參數調整工作。在控制台的「訓練」部分，您會看到「超參數調整工作」分頁，內容如下所示：



> 超參數調整工作大約需要 30 到 40 分鐘才能完成。

完成後，您就能按一下工作名稱，查看調整試驗的結果。



Hyperparameter tuning trials

Filter Enter property name or value

Trial ID	accuracy	Training step	learning_rate	momentum	num_neurons	Log
5	0.863	10	3.0998846321388662e-2	7.542808033097481e-1	512	View logs
8	0.863	10	1.5500957671284583e-2	9.048156494355999e-1	512	View logs
10	0.844	10	2.797941182764173e-2	1.9129967704805848e-1	128	View logs
12	0.828	10	2.1148856220143585e-2	-	128	View logs
15	0.816	10	1.582151532529689e-2	7.855723676903749e-1	128	View logs
13	0.727	10	3.6294410746151 15e-2	-	512	View logs
1	0.5	10	1.0000000000000002e-1	5e-1	128	View logs
2	0.5	10	1.4261924857754274e-1	7.125085901213717e-1	128	View logs
3	0.5	10	1.3541276772773478e-1	2.718250139251958e-1	512	View logs
4	0.5	10	4.3413513896765965e-1	9.54218711640798e-1	128	View logs

Rows per page: 10 1 – 10 of 15

🎉 恭喜！🎉

您已瞭解如何使用 Vertex AI 執行下列作業：

- 啟動超參數調整工作，以訓練自訂容器中提供的訓練程式碼。本範例使用 TensorFlow 模型，但您可以使用自訂容器，訓練以任何架構建構的模型。

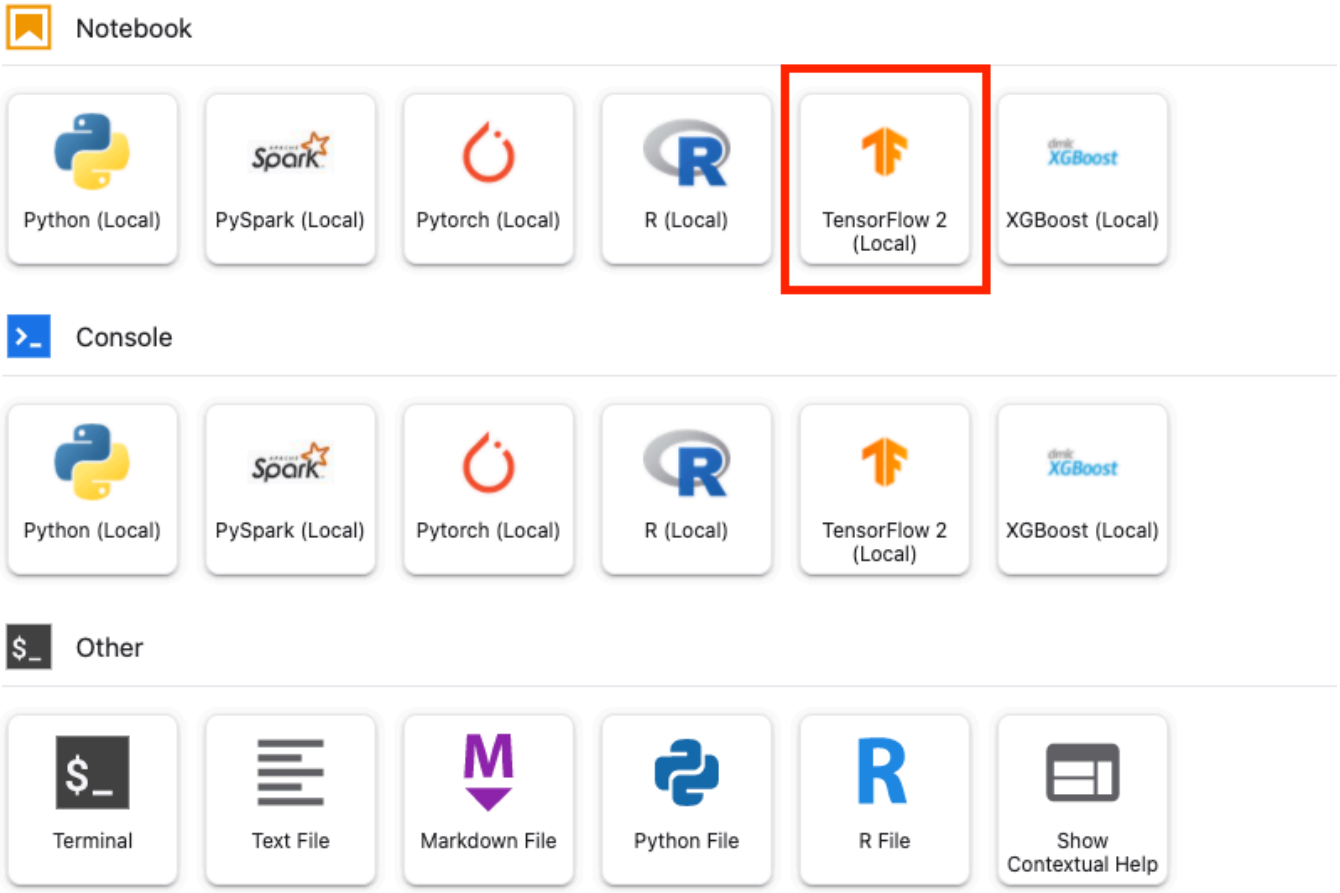
如要進一步瞭解 Vertex 的其他部分，請參閱說明文件。

步驟 6 · 約 50 分鐘

6. [選用] 使用 Vertex SDK

上一節說明如何透過 UI 啟動超參數調整工作。本節將說明如何使用 Vertex Python API，以其他方式提交超參數調整工作。

從 Launcher 建立 TensorFlow 2 筆記本。



匯入 Vertex AI SDK。

```
from google.cloud import aiplatform
from google.cloud.aiplatform import hyperparameter_tuning as hpt
```

如要啟動超參數調整工作，請先定義下列規格。您需要將 `image_uri` 中的 `{PROJECT_ID}` 換成您的專案。

```
# The spec of the worker pools including machine type and Docker image
# Be sure to replace PROJECT_ID in the `image_uri` with your project.

worker_pool_specs = [{
    "machine_spec": {
        "machine_type": "n1-standard-4",
        "accelerator_type": "NVIDIA_TESLA_V100",
        "accelerator_count": 1
    },
    "replica_count": 1,
    "container_spec": {
        "image_uri": "gcr.io/{PROJECT_ID}/horse-human:hypertune"
    }
}]


# Dictionary representing metrics to optimize.
# The dictionary key is the metric_id, which is reported by your training job,
# And the dictionary value is the optimization goal of the metric.
metric_spec={'accuracy':'maximize'}

# Dictionary representing parameters to optimize.
# The dictionary key is the parameter_id, which is passed into your training
# job as a command line argument,
# And the dictionary value is the parameter specification of the metric.
parameter_spec = {
    "learning_rate": hpt.DoubleParameterSpec(min=0.001, max=1, scale="log"),
    "momentum": hpt.DoubleParameterSpec(min=0, max=1, scale="linear"),
    "num_units": hpt.DiscreteParameterSpec(values=[64, 128, 512], scale=None)
}
```

接著，建立 `CustomJob`。您必須將 `{YOUR_BUCKET}` 換成專案中的 bucket，做為暫存空間。

```
# Replace YOUR_BUCKET
my_custom_job = aiplatform.CustomJob(display_name='horses-humans-sdk-job',
                             worker_pool_specs=worker_pool_specs,
                             staging_bucket='gs://{YOUR_BUCKET}')
```

然後建立並執行 `HyperparameterTuningJob`。

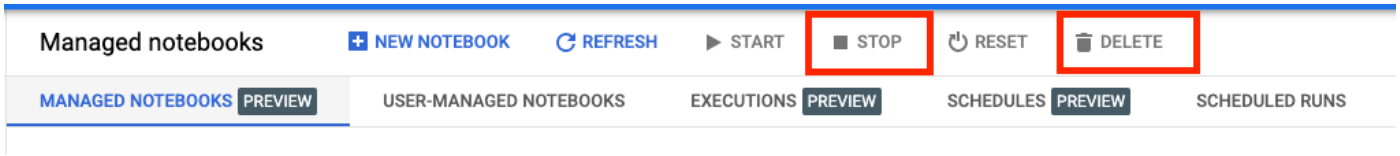
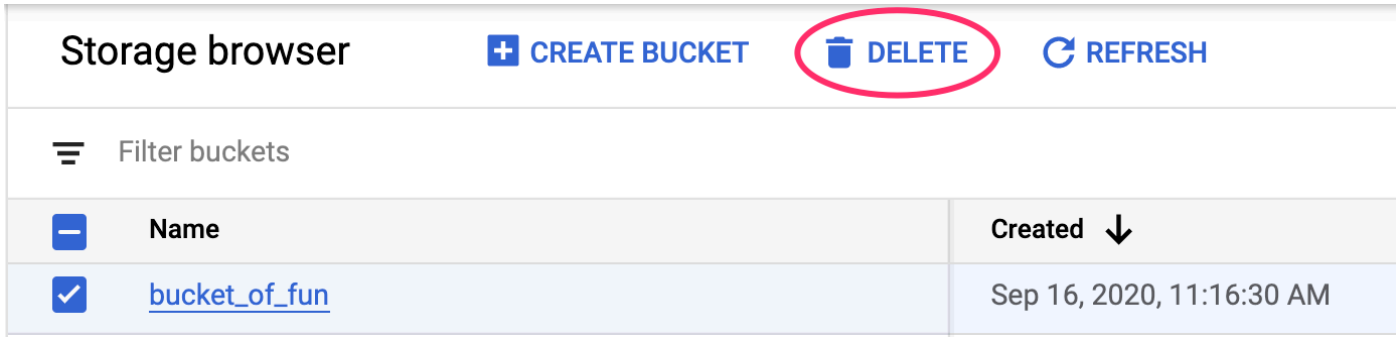
```
hp_job = aiplatform.HyperparameterTuningJob(
    display_name='horses-humans-sdk-job',
    custom_job=my_custom_job,
    metric_spec=metric_spec,
    parameter_spec=parameter_spec,
    max_trial_count=15,
    parallel_trial_count=3)

hp_job.run()
```

步驟 7 · 約 1 分鐘

7. 清除所用資源

由於我們將筆記本設定為在閒置 60 分鐘後逾時，因此不必擔心執行個體會關閉。如要手動關閉執行個體，請按一下控制台 Vertex AI Workbench 專區的「停止」按鈕。如要徹底刪除筆記本，請按一下「刪除」按鈕。



如要刪除 Storage Bucket，請使用 Cloud 控制台中的導覽選單瀏覽至 Storage，選取 bucket，然後按一下「Delete」：